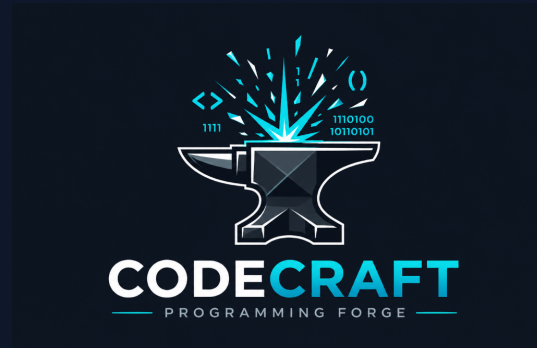




FINCONTROL

Sistema Web de Control de Dispositivos Financiados

Documentación Técnica — Stack, Arquitectura y Despliegue

Preparado por: CodeCraft Programming Forge

Versión: 1.0.0 | **Fecha:** Junio 2026

Entorno: Producción / Desarrollo Local

1. Resumen del Sistema

FinControl es una aplicación web full-stack diseñada para gestionar la venta de dispositivos móviles a crédito. Permite al administrador registrar ventas, hacer seguimiento de cuotas, visualizar calendarios de pago y enviar comandos de bloqueo/desbloqueo a dispositivos registrados. El sistema está dividido en dos capas independientes: un **Backend (API REST)** y un **Frontend SPA**.

Componente	Tecnología principal	Puerto / URL	Rol
Backend API	Node.js + TypeScript + Express	localhost:3000	Lógica de negocio, BD, comandos
Frontend SPA	React 18 + TypeScript + Vite	localhost:5173	Interfaz web de administración
Base de datos	PostgreSQL (Supabase)	Supabase Cloud / local	Persistencia de datos
Notificaciones Push	Firebase Admin SDK	FCM / APNs	Comandos a dispositivos móviles

2. Stack Tecnológico Completo

2.1 Backend — API REST

El servidor está escrito en **TypeScript** sobre **Node.js** usando el framework **Express.js**. Expone una API RESTful en la ruta base `/api/v1/` con tres módulos: autenticación, dispositivos y pagos.

Tecnología / Librería	Versión	Propósito
Node.js	≥ 18 LTS	Entorno de ejecución JavaScript del servidor
TypeScript	^5.5	Tipado estático, mayor seguridad en el código
Express.js	^4.19	Framework HTTP para enrutamiento y middlewares
PostgreSQL (pg)	^8.12	Driver para conectarse a la base de datos relacional

Firebase Admin SDK	^14.0	Envío de notificaciones push silenciosas (FCM/APNs)
jsonwebtoken (JWT)	^9.0	Autenticación stateless con tokens firmados
bcryptjs	^2.4	Hash seguro de contraseñas de administradores
dotenv	^16.4	Gestión de variables de entorno (.env)
cors	^2.8	Habilitación de peticiones cross-origin desde el frontend
ts-node-dev	^2.0	Recarga automática en desarrollo (hot reload)

Rutas de la API disponibles:

Método	Ruta	Descripción
GET	/health	Verificación de estado del servidor
POST	/api/v1/auth/login	Inicio de sesión del administrador (JWT)
GET	/api/v1/devices	Listar todos los dispositivos registrados
POST	/api/v1/devices	Registrar un nuevo dispositivo
POST	/api/v1/devices/:id/command	Enviar comando lock / unlock / wipe
GET	/api/v1/devices/:id/commands	Historial de comandos de un dispositivo
GET	/api/v1/devices/:id/pending	Comandos pendientes (polling del agente móvil)
POST	/api/v1/payments/confirm	Confirmar un pago recibido

2.2 Frontend — Single Page Application (SPA)

La interfaz de usuario es una **SPA** construida con **React 18** y **TypeScript**, empaquetada con **Vite 5** para desarrollo ultrarrápido. Los estilos utilizan **TailwindCSS** con componentes personalizados. La comunicación con el backend se realiza mediante **fetch API** nativo.

Tecnología / Librería	Versión	Propósito
React	^18.3	Librería UI basada en componentes y hooks
TypeScript	^5.2	Tipado estático en el frontend
Vite	^5.3	Bundler y servidor de desarrollo (HMR instantáneo)
TailwindCSS	^3.4	Framework de estilos utility-first
PostCSS + Autoprefixer	^8.4 / ^10.4	Procesamiento de CSS y compatibilidad de navegadores
Lucide React	^0.395	Librería de iconos SVG consistentes
localStorage API	nativo	Persistencia local de ventas y preferencias de tema
Fetch API	nativo	Peticiones HTTP al backend (sin librerías externas)

Vistas / Módulos del Frontend:

Vista	Ruta interna	Funcionalidad
Panel de Control	dashboard	Stats de créditos, consola de comandos, tabla de dispositivos
Ventas a Crédito	credits	Registro y seguimiento de ventas a crédito con cuotas
Calendario de Pagos	calendar	Vista mensual de cuotas programadas con indicadores visuales
Alertas de Mora	alerts	Listado de clientes con cuotas vencidas (+2 días = crítico)

3. Base de Datos — PostgreSQL / Supabase

El sistema utiliza **PostgreSQL** como base de datos relacional. Se puede alojar en **Supabase** (cloud gratuito) o en un servidor local. El esquema consta de dos tablas principales con índices de optimización.

Tabla: devices

Columna	Tipo	Descripción
id	SERIAL PK	Identificador único autoincremental
serial_number	VARCHAR(100) UNIQUE	Número de serie del dispositivo (obligatorio)
imei	VARCHAR(50)	IMEI del dispositivo (opcional)
brand	VARCHAR(50)	Marca del equipo (Samsung, Apple, etc.)
model	VARCHAR(100)	Modelo del equipo (Galaxy S24, iPhone 15...)
status	VARCHAR(20)	Estado: active locked suspended wiped
customer_name	VARCHAR(150)	Nombre del cliente propietario del crédito
customer_phone	VARCHAR(50)	Teléfono de contacto del cliente
device_token	TEXT	Token JWT de autenticación del hardware
last_sync_at	TIMESTAMP WITH TZ	Última vez que el dispositivo hizo check-in
created_at	TIMESTAMP WITH TZ	Fecha de registro del dispositivo

Tabla: commands

Columna	Tipo	Descripción
id	SERIAL PK	Identificador único
device_id	INTEGER FK	Referencia a devices.id (CASCADE ON DELETE)
type	VARCHAR(50)	Tipo de comando: lock unlock wipe
status	VARCHAR(20)	Estado: pending sent executed failed
signature	TEXT	Firma criptográfica asimétrica ECDSA del comando
token	VARCHAR(100)	Nonce único anti-replay por comando
payload	JSONB	Datos adjuntos, ej: { message: 'Realice su pago' }
executed_at	TIMESTAMP TZ	Marca de tiempo de ejecución en el dispositivo
created_at	TIMESTAMP TZ	Fecha de emisión del comando

Índices de rendimiento:

- **idx_devices_serial** → devices(serial_number) — búsqueda rápida por número de serie.
- **idx_commands_device_status** → commands(device_id, status) — consulta eficiente de comandos pendientes por dispositivo.

4. Estructura de Archivos del Proyecto

El repositorio está organizado en dos carpetas principales que mantienen separados el backend y el frontend, permitiendo desplegarlos de forma independiente.

```

bloqueio app/
  ■■■■ src/                                ← Backend (API)
  ■   ■■■■ server.ts                      ← Punto de entrada Express
  ■   ■■■■ config/                       ← Configuración DB y Firebase
  ■   ■■■■ controllers/
  ■     ■ ■■■■ authController.ts         ← Login / JWT
  ■     ■ ■■■■ deviceController.ts      ← CRUD dispositivos + comandos
  ■     ■ ■■■■ paymentController.ts     ← Confirmación de pagos
  ■   ■■■■ routes/
  ■     ■ ■■■■ authRoutes.ts
  ■     ■ ■■■■ deviceRoutes.ts
  ■     ■ ■■■■ paymentRoutes.ts
  ■   ■■■■ middlewares/                 ← Auth JWT middleware
  ■   ■■■■ db/                         ← Pool de conexión a PostgreSQL
  ■
  ■■■■ frontend/                       ← Frontend (SPA React)
  ■   ■■■■ index.html                  ← HTML base
  ■   ■■■■ vite.config.ts              ← Configuración Vite
  ■   ■■■■ tailwind.config.js          ← Configuración TailwindCSS
  ■   ■■■■ src/
  ■     ■ ■■■■ main.tsx                 ← Punto de entrada React
  ■     ■ ■■■■ App.tsx                  ← Componente raíz + todas las vistas
  ■     ■ ■■■■ index.css                 ← Variables CSS + estilos globales
  ■
  ■■■■ schema.sql                      ← DDL de la base de datos
  ■■■■ .env                            ← Variables de entorno (privado)
  ■■■■ firebase-service-account.json    ← Credenciales Firebase Admin
  ■■■■ package.json                    ← Dependencias del backend

```

5. Guía de Instalación y Montaje del Sistema

5.1 Requisitos Previos

Herramienta	Versión mínima	Descarga
Node.js	18 LTS o superior	nodejs.org
npm	9.x o superior	Incluido con Node.js
PostgreSQL	14 o superior (o cuenta Supabase gratuita)	postgresql.org / supabase.com
Git	2.x	git-scm.com
Editor	VS Code recomendado	code.visualstudio.com

5.2 Archivo de Variables de Entorno (.env)

Crea el archivo `.env` en la raíz del proyecto (junto a `package.json`) con el siguiente contenido:

```
# Puerto del servidor
PORT=3000
NODE_ENV=development

# Base de datos PostgreSQL
DB_HOST=db.tuproyecto.supabase.co
DB_PORT=5432
DB_NAME=postgres
DB_USER=postgres
DB_PASSWORD=tu_contrasena_segura

# JWT para autenticación de admins
JWT_SECRET=clave_secreta_muy_larga_y_segura_aqui

# Firebase (para notificaciones push)
FIREBASE_PROJECT_ID=tu-proyecto-firebase
```

5.3 Pasos de Instalación — Backend

Paso 1 — Instalar dependencias del backend

```
cd "bloqueo app"
npm install
```

Paso 2 — Crear la base de datos


```
# En Supabase: SQL Editor → New Query → pegar schema.sql
# En local: psql -U postgres -f schema.sql
```

Paso 3 — Iniciar el servidor en modo desarrollo

```
npm run dev
# Salida esperada:
# Servidor iniciado en el puerto 3000 (Entorno: development)
```

Paso 4 — Verificar que funciona

```
# Abrir en el navegador o con curl:
# GET http://localhost:3000/health
# Respuesta: { "status": "UP", "timestamp": "..."} }
```

5.4 Pasos de Instalación — Frontend

Paso 1 — Instalar dependencias del frontend

```
cd "bloqueo app/frontend"
npm install
```

Paso 2 — Iniciar el servidor de desarrollo

```
npm run dev
# Salida esperada:
# VITE v5.x.x ready in 300 ms
# ➔ Local: http://localhost:5173/
```

Paso 3 — Abrir la interfaz

```
# Navegar a: http://localhost:5173
# El frontend se conecta automáticamente al backend en localhost:3000
```

6. Despliegue en Producción (Servidor VPS)

Para producción se recomienda un servidor VPS con Ubuntu 22.04. El backend se ejecuta como servicio con PM2, el frontend se construye como archivos estáticos y se sirve con Nginx.

Componente	Herramienta	Descripción
Proceso Backend	PM2	Gestor de procesos Node.js. Reinicio automático al fallar.
Servidor Web	Nginx	Reverse proxy para el backend y servidor de archivos estáticos.
SSL/HTTPS	Certbot (Let's Encrypt)	Certificado SSL gratuito y renovación automática.
Base de datos	Supabase / Render	PostgreSQL gestionado en la nube (opción más económica).
CI/CD	GitHub Actions (opcional)	Deploy automático al hacer push a la rama main.

Comandos de despliegue en VPS:

```
# 1. Clonar el repositorio
git clone https://github.com/tu-usuario/fincontrol.git /var/www/fincontrol

# 2. Instalar dependencias backend y compilar
cd /var/www/fincontrol
npm install && npm run build

# 3. Compilar el frontend
cd frontend && npm install && npm run build

# 4. Configurar variables de entorno
cp .env.example .env && nano .env

# 5. Iniciar el backend con PM2
npm install -g pm2
pm2 start dist/server.js --name fincontrol-api
pm2 save && pm2 startup

# 6. Configurar Nginx (ver configuración en docs/)
sudo nginx -t && sudo systemctl reload nginx
```

Configuración básica de Nginx:

```
server {  
    listen 80;  
    server_name tudominio.com;  
  
    # Frontend: archivos estáticos  
    root /var/www/fincontrol/frontend/dist;  
    index index.html;  
    try_files $uri $uri/ /index.html;  
  
    # Backend: proxy inverso  
    location /api/ {  
        proxy_pass http://localhost:3000;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
    }  
}
```

7. Resumen Visual del Stack Completo

Tabla de referencia rápida con todas las tecnologías del sistema, agrupadas por categoría:

Categoría	Tecnología	Lenguaje / Formato	Rol en el sistema
Backend Runtime	Node.js 18 LTS	JavaScript	Entorno servidor
Backend Language	TypeScript 5.5	TypeScript	Código tipado del servidor
Backend Framework	Express.js 4	TypeScript/JS	API REST y rutas HTTP
Auth	JWT (jsonwebtoken)	TypeScript	Autenticación stateless
Hashing	bcryptjs	TypeScript	Hash seguro de contraseñas
Push Notifications	Firebase Admin SDK	TypeScript	Comandos a dispositivos móviles
Frontend Runtime	Vite 5 + React 18	TSX / HTML	SPA + bundler de desarrollo
Frontend Language	TypeScript 5.2	TypeScript	Código tipado del cliente
UI Styles	TailwindCSS 3.4	CSS / PostCSS	Estilos utility-first
UI Icons	Lucide React	TSX / SVG	Iconografía consistente
Base de datos	PostgreSQL 14+	SQL	Persistencia relacional
DB Hosting	Supabase / local	SQL	Base de datos en la nube
Schema	schema.sql	SQL (DDL)	Definición de tablas e índices
Variables de entorno	dotenv	.env	Secretos y configuración
Proceso producción	PM2	JSON / Shell	Gestor de procesos Node.js
Web Server prod	Nginx	nginx.conf	Reverse proxy + archivos estáticos

SSL/HTTPS	Let's Encrypt	Certbot CLI	Certificado TLS gratuito
-----------	---------------	-------------	--------------------------

8. Scripts npm Disponibles

Backend (raíz del proyecto):

Script	Comando	Descripción
npm run dev	ts-node-dev --respawn src/server.ts	Inicia el servidor con hot-reload (desarrollo)
npm run build	tsc	Compila TypeScript a JavaScript en /dist
npm start	node dist/server.js	Inicia el servidor compilado (producción)

Frontend (carpeta /frontend):

Script	Comando	Descripción
npm run dev	vite	Inicia Vite con HMR en localhost:5173 (desarrollo)
npm run build	tsc && vite build	Compila TS y genera archivos estáticos en /dist
npm run preview	vite preview	Vista previa del build de producción localmente

Documento generado automáticamente por CodeCraft Programming Forge — Junio 2026. Para actualizaciones o soporte técnico, contactar al equipo de desarrollo.